

MÓDULO 8: SEGURIDAD

PRINCIPIOS DE "SECURITY BY DESIGN" Y "SECURITY BY DEFAULT"

INTRODUCCIÓN AL DESARROLLO SEGURO

BIG school



Security by Design y Security by Default

BIG school

Índice

- Introducción
- Origen del problema
- Nacimiento del concepto
- Desde cuándo se habla de metodologías seguras
- Por qué no se usaban — y aún no se usan del todo
- Por qué cuesta tanto la adopción
- La IA entra en escena — y complica las cosas
- Cómo la IA puede ayudarnos a escribir código seguro
- Conclusiones
- El código: donde empieza la seguridad
- Ejemplos



Security by Design y Security by Default

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Introducción

La industria solía enfocarse solo en que el software funcionara y se vendiera, dejando la seguridad para después.

Ahora, el reto es diseñar pensando en proteger desde la primera etapa.

Origen del problema

En los años 90 y principios de los 2000 la ciberseguridad no era prioridad.

Con la digitalización, los ataques se volvieron costosos y peligrosos.

La pregunta clave: ¿por qué nuestros sistemas son tan fáciles de atacar?

Nacimiento del concepto

Así nació Security by Design:

"La seguridad no puede ser un parche. Tiene que estar en el ADN del producto."

Security by Default complementa la filosofía:

"Las configuraciones por defecto deben ser seguras, aunque el usuario no cambie nada. "

Desde cuándo se habla de metodologías seguras

Marcos formales comenzaron alrededor de 2004 con el SDL de Microsoft.

Iniciativas como OWASP y NIST promueven integrar seguridad desde el principio.

El resultado: menos riesgos y menos costes.

Por qué no se usaban — y aún no se usan del todo

La seguridad no genera ingresos inmediatos y requiere cultura y conocimiento.

Muchas empresas priorizan la velocidad sobre la seguridad. Por eso aún hay bases de datos expuestas y software inseguro.

Por qué cuesta tanto la adopción

Cambiar el mindset es difícil:

La seguridad debe ser parte del diseño, no un “extra”.

Muchos desarrolladores aprenden sobre seguridad solo tras un incidente.

La IA entra en escena — y complica las cosas

La IA ayuda a generar código y detectar errores, pero también puede reproducir vulnerabilidades.

La seguridad debe acompañar el avance tecnológico, nunca dejarlo solo.

Cómo la IA puede ayudarnos a escribir código seguro

La IA puede:

- Revisar código y detectar patrones inseguros.
- Automatizar auditorías y sugerir correcciones seguras.

El reto es usar la IA como copiloto, no como único piloto.

Conclusiones

1. **Security by Design y Security by Default son una filosofía de desarrollo.**
1. **La confianza del usuario se gana desde el diseño seguro, no con parches tardíos.**
1. **En este módulo aprenderás a construir software que resista y proteja, usando IA y buenas prácticas como base.**

El código: donde empieza la seguridad

Una línea de código puede ser el punto de entrada de todo un ataque.

Ejemplo 1: Inyección SQL (código inseguro)

```
usuario = input("Usuario: ")  
password = input("Contraseña: ")
```

```
query = f"SELECT * FROM usuarios WHERE user='{usuario}' AND pass='{password}'"  
cursor.execute(query)
```

A simple vista parece correcto, pero es una invitación abierta al ataque. Si alguien usa: ' OR '1'='1 la consulta devuelve todos los usuarios.

Autenticación bypass: entra sin conocer ninguna contraseña.

Por qué ocurre esta falla

Funciona durante las pruebas.

El desarrollador no pensó en ataques maliciosos.

"Ya lo arreglaremos después" se convierte en excusa.

Código seguro con parametrización

```
usuario = input("Usuario: ")  
password = input("Contraseña: ")
```

```
query = "SELECT * FROM usuarios WHERE user=%s AND  
pass=%s"  
cursor.execute(query, (usuario, password))
```

La parametrización indica que esto es un valor, no una instrucción SQL. Así el atacante controla solo texto, no código malicioso.

Esta es una aplicación práctica del Security by Design en el código.

Reflexión breve

Security by Design no castiga, cambia la forma de pensar:

“¿Qué pasa si alguien intenta romper esto?”

Preguntar a tiempo ahorra millones en incidentes.

Ejemplo 2: Manejo inseguro de contraseñas

Código inseguro:

```
# Almacenar contraseñas (¡mal ejemplo!)  
password = input("Contraseña: ")  
guardar_en_db(password)
```

A veces incluso con un hash inseguro:

```
hashed = hashlib.md5(password.encode()).hexdigest()  
guardar_en_db(hashed)
```

- MD5 está roto hace años.
- Sin salting, contraseñas iguales generan hashes iguales.
- Son vulnerables a diccionarios precomputados.

Manejo seguro de contraseñas

```
import bcrypt
```

```
password = input("Contraseña: ")  
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())  
guardar_en_db(hashed)
```

Y al verificar:

```
bcrypt.checkpw(password.encode(), hashed)
```

- Usa sal y hashing fuerte.
- Aunque roben la DB es muy difícil romper el hash.

Security by Design en el producto completo

No solo en código: empieza en el diseño de producto.

Preguntas clave:

1. ¿Qué datos sensibles manejará?
2. ¿Quién puede acceder?
3. ¿Qué pasa si falla un componente?
4. ¿Cómo se detectan y responden incidentes?

Metodologías como STRIDE o PASTA ayudan a identificar y documentar riesgos previos a un ataque.

Ejemplo práctico: diseño de una API

API insegura:

GET /api/users?id=123

Atacante prueba:

/api/users?id=124

/api/users?id=125

Y accede a toda la DB (IDOR).

Solución segura

```
@app.route("/api/users/<id>")
def get_user(id):
    user = get_current_user()
    if not user or not user.can_access(id):
        return {"error": "Unauthorized"}, 403

    data = db.get_user_by_id(id)
    return sanitize_output(data)
```

Así el diseño incluye autorización y sanitización.

Security by Default

Aunque el código esté bien escrito, el sistema puede ser inseguro si la configuración por defecto no es segura.

Ejemplo: instalación con usuario admin y contraseña admin123.

Reflexión final

Security by Design y Security by Default no son solo normas, sino una filosofía.

“No construyas algo que funcione. Construye algo que resista.”

Aprenderemos a escribir, revisar y desplegar software pensando como hackers, actuando como ingenieros y diseñando como arquitectos.